

Reverse Engineering SaccadeJEPA: From Joint-Embedding Predictive Architectures to Gaze Dynamics Simulation

Emad Bahreinipour

May 2026

Contents

1	Joint-Embedding Predictive Architectures	3
1.1	Motivation	3
1.2	Limitations of Existing Paradigms	3
1.3	Core Principle: Prediction in Representation Space	3
1.4	Preventing Representation Collapse	4
1.5	Toward Hierarchical World Models	4
2	I-JEPA: Image-Based JEPA	4
2.1	From Theory to Images: Overview	4
2.2	Patch Tokenisation	5
2.3	Multi-Block Masking Strategy	5
2.4	Encoder Architecture	6
2.5	Predictor Architecture	6
2.6	Training Objective	7
2.7	Absence of Hand-Crafted Augmentations	7
2.8	Properties of Learned Representations	7
2.9	Relevance to Gaze Modelling	8
3	SaccadeJEPA: Adaptation for Attentive Vision	8
3.1	Architectural Overview	8
3.2	Saccade Generation	9
3.3	Affine Embedding	9
3.4	Forward Pass	10
3.5	Training Losses	10
3.6	Comparison with Standard I-JEPA	11
3.7	Relationship to Biological Gaze	11
3.8	Assessment for Gaze Simulation Reuse	12
4	A Less Obscure Reuse of SaccadeJEPA on FIND	12
4.1	Motivation	12
4.2	The Architectural Substitution	13
4.3	Training on FIND	15

4.4	Results	16
4.5	Discussion	17
4.6	Limitations	19

1 Joint-Embedding Predictive Architectures

1.1 Motivation

A central open problem in artificial intelligence is the gap between how efficiently humans learn about the world and the data-hungry nature of current machine-learning systems. A human infant acquires an intuitive understanding of physics, object permanence, and spatial relationships from a few months of passive observation, whereas state-of-the-art models typically require millions of labelled examples to achieve comparable competence on narrow tasks.

LeCun (2022) argues that this gap stems from the absence of *internal world models* in mainstream architectures and proposes a blueprint for self-supervised learning called the **Joint-Embedding Predictive Architecture (JEPA)**. The key idea is to abandon the prediction of raw sensory data and instead learn to *predict in an abstract representation space*, thereby allowing the model to discard irrelevant detail and focus on the semantic structure of its inputs.

1.2 Limitations of Existing Paradigms

JEPA is positioned as a response to three dominant self-supervised paradigms, each of which suffers from a characteristic weakness:

Generative models. Architectures such as variational autoencoders and diffusion models are trained to reconstruct every detail of the input signal. In the visual domain this means predicting exact pixel values, including perceptually irrelevant variations in texture, lighting, and noise. This forces the model to allocate capacity to information that contributes little to downstream semantic understanding.

Contrastive methods. Approaches such as SimCLR and CLIP learn representations by attracting embeddings of similar inputs and repelling embeddings of dissimilar ones. While effective, they require carefully constructed sets of *negative samples*. As the complexity of the data grows, the number of negatives needed increases combinatorially, making the training signal noisy and expensive to produce.

Reinforcement learning. Pure trial-and-error learning is extremely sample-inefficient and, on its own, is insufficient for acquiring the rich predictive world models that underpin human cognition.

1.3 Core Principle: Prediction in Representation Space

The central insight of JEPA can be stated concisely: rather than mapping an input to a reconstruction of missing data in the *input* space, the model maps both the observed and the missing portions of the input into a shared *representation* space and performs prediction there.

Formally, let $\mathbf{x}$ denote the observed portion of an input and $\mathbf{y}$ the portion to be predicted. Two encoders map these to representations:

$$s_x = f_\theta(\mathbf{x}), \tag{1}$$

$$s_y = f_{\bar{\theta}}(\mathbf{y}), \tag{2}$$

where f_θ is the *context encoder* (trained by gradient descent) and $f_{\bar{\theta}}$ is the *target encoder* (updated via an exponential moving average of θ). A predictor network then estimates the target representation from the context:

$$\hat{s}_y = g_\phi(s_x), \quad (3)$$

and the training objective minimises the distance between $\hat{s}_y$ and s_y :

$$\mathcal{L} = d(\hat{s}_y, s_y), \quad (4)$$

where $d(\cdot, \cdot)$ is typically the mean squared error.

Because the loss operates in representation space, the encoders are free to discard unpredictable, low-level variation and retain only the information that is both *predictable* and *useful*—a property that is impossible to achieve when the training target is a raw pixel reconstruction.

1.4 Preventing Representation Collapse

A critical challenge in any joint-embedding framework is *representation collapse*: the trivial solution in which both encoders map every input to the same constant vector, yielding zero loss but carrying no information. Different families of methods address collapse in different ways. Contrastive methods introduce explicit repulsion through negative samples; redundancy-reduction methods (e.g. Barlow Twins) decorrelate representation dimensions; JEPA follows the strategy pioneered by BYOL and introduces *architectural asymmetry*.

Concretely, the target encoder parameters $\bar{\theta}$ are not updated by gradient descent but by an exponential moving average (EMA) of the context encoder parameters:

$$\bar{\theta} \leftarrow \tau \bar{\theta} + (1 - \tau) \theta, \quad (5)$$

with momentum coefficient $\tau \in [0.996, 0.999]$. This makes the target encoder a slowly evolving reference: stable enough to provide a consistent learning signal, yet non-trivial enough that the context encoder and predictor must learn genuinely informative representations to track it.

1.5 Toward Hierarchical World Models

Beyond a single predictive module, LeCun envisions stacking multiple JEPA layers into a hierarchy. Lower levels would capture short-term, fine-grained predictions (edges, textures, local motion), while higher levels would operate over longer time scales and more abstract quantities (object identities, goals, plans). This hierarchical JEPA serves as a theoretical blueprint for autonomous agents that learn world models capable of planning and reasoning—though full implementations of this vision remain an open research direction.

2 I-JEPA: Image-Based JEPA

2.1 From Theory to Images: Overview

While Section 1 described JEPA as a general theoretical framework, **I-JEPA** (Image-based JEPA), introduced by Assran et al. (2023), is its first concrete realisation for the

visual domain. I-JEPA instantiates every component of the abstract blueprint—context encoder, target encoder, predictor, and masking strategy—using Vision Transformers and a carefully designed multi-block masking scheme. The result is a self-supervised method that learns rich, semantic image representations *without* pixel-level reconstruction and *without* hand-crafted data augmentations.

2.2 Patch Tokenisation

Following the Vision Transformer (ViT) paradigm, I-JEPA converts a raw image into a sequence of discrete tokens that a Transformer can process:

1. An input image of size $H \times W$ pixels is divided into a regular grid of non-overlapping patches, each of size $p \times p$ pixels (typically $p = 16$).
2. Each patch is flattened and linearly projected into a D -dimensional embedding vector.
3. Learnable positional embeddings are added so that the model retains information about the spatial location of each patch on the grid.

For a 224×224 image with $p = 16$, this yields a $14 \times 14 = 196$ token grid. All subsequent operations—encoding, masking, and prediction—are defined over this grid rather than over raw pixels.

2.3 Multi-Block Masking Strategy

The masking strategy is central to I-JEPA and differs markedly from other masked-image modelling approaches such as MAE. Two complementary sets of regions are sampled from the patch grid for each training image:

Target blocks. N_t rectangular regions (typically $N_t = 4$) are sampled independently. Each block is parameterised by a *scale* s_t (the fraction of total patches it covers, uniformly sampled from $[0.15, 0.2]$) and an *aspect ratio* r_t (sampled from $[0.75, 1.5]$). The resulting blocks are spatially contiguous regions of the patch grid whose positions are chosen at random. These blocks define the portions of the image whose *representations* the model must learn to predict.

Context block. A single large rectangular region is sampled from the grid with a comparatively large scale. All patches that fall inside any target block are then *removed* from this region. The remaining patches constitute the context—typically between 40% and 60% of the total grid. Crucially, context and target patches **never overlap**: the model receives no direct information about the regions it is asked to predict.

Why large, contiguous blocks? This design choice is deliberate. In MAE, individual patches are masked at random positions across the image; because nearby visible patches are almost always available, the decoder can reconstruct missing pixels through local interpolation without needing a deep semantic understanding. By contrast, I-JEPA masks entire contiguous regions and predicts their *abstract representations*. Recovering the representation of a large missing area requires the encoder to develop a global, semantic understanding of the scene—local texture cues are insufficient.

2.4 Encoder Architecture

I-JEPA employs two Vision Transformer encoders that share the same architecture (number of layers, embedding dimension, attention heads) but differ in their roles and update rules:

Context encoder f_θ . Receives *only* the context patch tokens (with their positional embeddings) and processes them through a standard stack of Transformer blocks (multi-head self-attention followed by feed-forward layers). The output is a set of representation vectors $\{s_{x_i}\}_{i=1}^{N_c}$, one per context patch. This encoder is trained end-to-end via gradient descent.

Target encoder $f_{\bar{\theta}}$. Receives *only* the target patch tokens and produces a set of target representations $\{s_{y_j}\}_{j=1}^{N'_t}$, where N'_t is the total number of patches across all target blocks. No gradient flows through this encoder; its parameters are updated exclusively by the exponential moving average rule defined in Equation (5). As discussed in Section 1.4, this asymmetry is the primary mechanism that prevents representation collapse.

2.5 Predictor Architecture

The predictor g_ϕ is a *narrow* Transformer—fewer layers and a smaller hidden dimension than the encoders—that bridges the gap between what the model has seen and what it must infer. Its operation proceeds in three steps:

1. **Input assembly.** The context representations $\{s_{x_i}\}$ are concatenated with a set of learnable **mask tokens**, one for each target patch. Each mask token is equipped with the *positional embedding* of the target patch it represents. This way the predictor knows *where* in the image it must predict, even though it has no content information about those locations.
2. **Transformer processing.** The combined sequence of context representations and positioned mask tokens is processed through the predictor’s Transformer layers. Self-attention allows each mask token to attend to all context representations (gathering evidence from visible regions) and to other mask tokens (ensuring coherence across the predicted region).
3. **Prediction extraction.** After the final layer, only the output vectors at the mask-token positions are retained. These vectors $\{\hat{s}_{y_j}\}_{j=1}^{N'_t}$ are the predicted target representations.

The predictor is kept deliberately small. A high-capacity predictor could learn to approximate the target representations on its own, allowing the context encoder to produce weak, uninformative features. By constraining the predictor’s capacity, I-JEPA ensures that the context encoder must bear the primary responsibility for building rich representations—the representations that will ultimately be used for downstream tasks.

2.6 Training Objective

The loss is the mean squared error between the predicted and actual target representations, averaged over all target patches:

$$\mathcal{L} = \frac{1}{N'_t} \sum_{j=1}^{N'_t} \|\hat{s}_{y_j} - s_{y_j}\|_2^2. \quad (6)$$

Gradients of $\mathcal{L}$ are back-propagated through the predictor g_ϕ and the context encoder f_θ ; the target encoder $f_{\bar{\theta}}$ is updated solely via EMA (Equation 5).

2.7 Absence of Hand-Crafted Augmentations

Most self-supervised visual methods rely on aggressive data augmentations—random cropping, colour jittering, horizontal flipping, Gaussian blurring—to create multiple views of the same image and learn invariant representations. While effective, this approach hard-codes the researcher’s assumptions about which transformations should be irrelevant. For example, enforcing colour invariance helps image classification but harms any task in which colour is discriminative.

I-JEPA uses **no hand-crafted augmentations**. The only source of variability across training iterations is the random masking itself. Because the loss is computed in representation space, the encoders are free to learn whichever invariances are genuinely useful for prediction, rather than those imposed externally. Empirically, this produces representations that transfer well across a broader range of downstream tasks than augmentation-heavy alternatives.

2.8 Properties of Learned Representations

The representations learned by I-JEPA occupy a distinctive position in the landscape of self-supervised features:

- **Compared with pixel-reconstruction methods (e.g. MAE).** Because MAE reconstructs raw pixels, it tends to learn low-level, texture-oriented features. I-JEPA, predicting in representation space, learns more *semantic* features and consequently requires fewer labelled examples during fine-tuning to reach equivalent performance.
- **Compared with contrastive methods (e.g. DINO, iBOT).** Contrastive methods with heavy augmentation learn highly global, invariant features that excel at classification but discard spatial and local structure. I-JEPA retains more spatial information while still being semantic, which improves performance on spatially demanding tasks such as object detection and depth estimation.

In summary, I-JEPA provides a balanced trade-off: its representations are abstract enough for high-level recognition yet spatially detailed enough for dense prediction—a property that is directly relevant to the gaze-modelling extension discussed in Section 3.

2.9 Relevance to Gaze Modelling

Several design elements of I-JEPA have a natural correspondence with mechanisms of biological visual attention:

- **Masking as gaze selection.** Choosing which patches the context encoder observes is analogous to choosing where the eye fixates; the unobserved target blocks correspond to peripheral regions whose content must be inferred.
- **Prediction as peripheral inference.** The predictor’s task—estimating the representation of unseen regions from the observed ones—mirrors the brain’s ability to construct a coherent percept from a sparse sequence of fixations.
- **Positional mask tokens as spatial attention.** The predictor explicitly encodes *where* in the image it must predict, providing a spatial-attention signal akin to the target selection that precedes a saccade.

The SACCADEJEPA variant examined in the next section builds on these correspondences by replacing the random, single-shot masking of I-JEPA with a *sequential, dynamic* masking process that more closely mirrors the temporal structure of human gaze: saccades followed by fixations, progressively accumulating information about a scene.

3 SaccadeJEPA: Adaptation for Attentive Vision

The repository at github.com/LumenPallidium/jepa provides, alongside a standard I-JEPA implementation (ViTJepa), a variant called **SaccadeJepa**. Importantly, SaccadeJepa is the repository author’s *own, unpublished* variant—it is not a reproduction of a published method. Architecturally, it is *not* a subclass of the base JEPA hierarchy; it is a standalone `nn.Module` with its own architecture, forward pass, and training losses. This section presents the results of a systematic reverse engineering of the codebase and assesses the extent to which SaccadeJepa implements a genuine visual-attention mechanism.

3.1 Architectural Overview

Whereas I-JEPA operates on a grid of patch tokens and learns by predicting the representations of *masked* patches, SaccadeJepa operates on *full image crops* and learns by predicting how an image representation changes under a small spatial perturbation. The perturbation—a random rotation and translation applied in pixel space—serves as a minimal analogue of an involuntary eye movement (microsaccade). The model’s task is to predict the target encoder’s representation of the *perturbed* view given the context encoder’s representation of the *original* view, plus a description of the applied perturbation.

The main components are:

1. **SaccadeCropper** (`saccade.py`) — generates two views of the input image: the original centre crop and a spatially perturbed centre crop, together with the affine parameters that describe the perturbation.
2. **Context encoder** — a ConvNeXt-Tiny backbone (from torchvision) that encodes View 1 (the original crop) into a 1536-dimensional vector.

3. **Target encoder** — a deep copy of the context encoder, frozen and updated exclusively via EMA, that encodes View 2 (the perturbed crop).
4. **Affine embedder** — a NeRF-style sinusoidal positional encoding followed by a learned linear projection that maps the 3-dimensional affine parameters (1 rotation + 2 translations) to a 1536-dimensional vector.
5. **Predictor** — a 3-layer MLP (default) or, optionally, a Transformer, that takes the sum of the context representation and the affine embedding and produces a prediction of the target representation.

3.2 Saccade Generation

The SaccadeCropper produces a perturbed view in three steps:

Step 1: Sample affine parameters. For each image in the batch a rotation angle α and a 2D translation vector $\mathbf{t}$ are drawn independently from uniform distributions:

$$\alpha \sim \mathcal{U}(-\alpha_{\max}, +\alpha_{\max}), \quad \alpha_{\max} = 2^\circ \approx 0.035 \text{ rad}, \quad (7)$$

$$\mathbf{t} \sim \mathcal{U}(-t_{\max}, +t_{\max})^2, \quad t_{\max} = \frac{\max_translation}{H_{\text{in}}}, \quad (8)$$

where H_{in} is the full input height (default 200 px) and $\max_translation = \lfloor 0.9 (H_{\text{in}} - H_{\text{crop}}) \rfloor$. With default parameters this yields a maximum translation of roughly $\pm 32\%$ of the image extent.

The sampling is **purely random**: it is not conditioned on image content, not learned, and not guided by any saliency or attention signal.

Step 2: Apply the affine transform. An affine matrix is constructed from α and $\mathbf{t}$ and applied to the full-size input image via bilinear interpolation (`torch.nn.functional.affine_grid + grid_sample`).

Step 3: Centre crop both views. Both the original and the transformed image are centre-cropped to the target resolution (128×128 by default). This centre crop represents the “fixation region”—the portion of the visual field that is actually processed by the encoders.

3.3 Affine Embedding

The three affine parameters are embedded into the encoder’s representation space using a NeRF-inspired sinusoidal encoding. For a scalar input v , the encoding at L frequency levels is:

$$\gamma(v) = [\sin(\pi 2^0 v), \cos(\pi 2^0 v), \dots, \sin(\pi 2^{L-1} v), \cos(\pi 2^{L-1} v)]. \quad (9)$$

The translation components use $L = 4$ (producing 16 features) and the rotation uses $L = 10$ (producing 20 features), for a total of 36 dimensions. A learned linear layer projects this 36-dimensional vector to the encoder’s output dimension (1536), and the result is *added* element-wise to the context representation before it enters the predictor.

This embedding serves the same functional role as the positional mask tokens in I-JEPA: it tells the predictor *how* the two views differ spatially, so that the predictor can focus on learning the *content-dependent* change in representation.

3.4 Forward Pass

A complete forward pass proceeds as follows:

1. **Generate saccade:** $(\mathbf{x}_1, \mathbf{x}_2, \mathbf{a}) = \text{SaccadeCropper}(\mathbf{x})$, where $\mathbf{x}_1$ is the perturbed crop, $\mathbf{x}_2$ the original crop, and $\mathbf{a} = (\alpha, t_x, t_y)$.
2. **Random view swap** (50% probability): the two views are exchanged and the sign of the affine parameters is flipped, serving as a data-augmentation strategy.
3. **Context encoding:** $s_x = f_\theta(\mathbf{x}_1)$ (with gradient).
4. **Target encoding:** $s_y = f_{\bar{\theta}}(\mathbf{x}_2)$ (no gradient; EMA-updated).
5. **Affine embedding:** $e_a = \text{affine_embedder}(\mathbf{a})$.
6. **Prediction:** $\hat{s}_y = g_\phi(s_x + e_a)$.
7. **Optional cycle consistency:** $\hat{s}_x = g_\phi(\hat{s}_y + \text{affine_embedder}(-\mathbf{a}))$, enforcing that the predictor can “undo” the transform.

Exactly **one** saccade is generated per image per forward pass. There is no mechanism for sequential fixations, no recurrence, and no accumulation of information across multiple viewing steps.

3.5 Training Losses

The total training loss for SaccadeJepa comprises three (optionally four) components:

Prediction loss. The primary objective is to minimise the Huber loss between the predicted and actual target representations:

$$\mathcal{L}_{\text{pred}} = \text{HuberLoss}(\hat{s}_y, s_y). \quad (10)$$

The choice of Huber loss (rather than MSE as in I-JEPA) provides robustness to outlier representations that may arise from large spatial perturbations.

Cycle-consistency loss. When enabled, the predictor is additionally asked to invert the transformation by predicting the original context from the predicted target:

$$\mathcal{L}_{\text{cycle}} = \|\hat{s}_x - s_x\|_2^2, \quad \text{weight } \lambda_{\text{cycle}} = 0.25. \quad (11)$$

VICReg regularisation. To prevent representation collapse, SaccadeJepa employs the variance and covariance terms from VICReg:

$$\mathcal{L}_{\text{var}} = \frac{1}{D} \sum_{d=1}^D \max(0, \gamma - \sqrt{\text{Var}(s_x^{(d)}) + \epsilon}), \quad \gamma = 4, \quad (12)$$

$$\mathcal{L}_{\text{cov}} = \frac{1}{D} \sum_{i \neq j} [\text{Cov}(s_x)]_{ij}^2, \quad (13)$$

with weights $\lambda_{\text{var}} = 25$ and $\lambda_{\text{cov}} = 1$. The variance term ensures that each representation dimension maintains sufficient spread, while the covariance term decorrelates dimensions.

Affine prediction loss (optional). An alternative training mode replaces the affine embedding with an auxiliary head that must *predict* the applied affine from the pair of representations. This introduces cosine-distance and magnitude-difference losses on the affine parameters.

Total loss.

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda_{\text{cycle}} \mathcal{L}_{\text{cycle}} + \lambda_{\text{var}} \mathcal{L}_{\text{var}} + \lambda_{\text{cov}} \mathcal{L}_{\text{cov}}. \quad (14)$$

3.6 Comparison with Standard I-JEPA

Table 1 summarises the key architectural and methodological differences between the standard I-JEPA implementation in the repository (ViTJepa) and SaccadeJepa.

Table 1: Summary of differences between I-JEPA and SaccadeJepa as implemented in the repository.

Aspect	I-JEPA (ViTJepa)	SaccadeJepa
Core task	Predict masked-patch representations	Predict representation after spatial perturbation
Input views	1 image $\rightarrow$ context + target blocks	2 centre crops: original vs. rotated/translated
Masking	Random rectangular blocks on patch grid	None; both views are complete crops
Backbone	Custom ViT (Transformer on patches)	ConvNeXt-Tiny (CNN on full crops)
Predictor	Transformer (same width as encoder)	3-layer MLP (1536 $\rightarrow$ 1536)
Positional info	Learnable per-patch embeddings	NeRF-style sinusoidal encoding of affine parameters
Prediction loss	MSE	Huber
Collapse prevention	Decorrelation (weight 0.01)	VICReg (variance wt. 25, covariance wt. 1)
Additional losses	—	Cycle consistency, optional affine prediction loss
Biological metaphor	Filling in missing visual regions	Predicting consequences of eye movement

3.7 Relationship to Biological Gaze

The README of the repository motivates SaccadeJepa with the observation that “*the eyes are constantly jittering and internally the brain predicts how this will affect the visual scene.*” This is a reference to fixational eye movements—involuntary, small-amplitude movements (tremor, drift, microsaccades) that occur during nominal fixation.

The following elements of SaccadeJepa have a defensible biological analogue:

- **Random spatial perturbation $\rightarrow$ fixational eye movements.** The uniform random rotation and translation loosely mirror the stochastic nature of microsaccades and ocular drift.
- **Predictive coding.** The core training objective—predicting the sensory consequences of a movement before executing it—is a central tenet of predictive-coding theories of the brain.
- **Two-view paradigm.** The pairing of a pre-perturbation and a post-perturbation view echoes the pre-saccadic and post-saccadic retinal images that the visual system must reconcile.

However, several aspects that are fundamental to biological gaze dynamics are absent from the current implementation:

- **No sequential fixations.** Only a single saccade is produced per image. Real visual exploration involves a sequence of 3–5 fixations per second, each to a new location.
- **No saliency-driven gaze control.** Saccade targets are sampled uniformly at random. Biological saccades are guided by both bottom-up saliency (contrast, edges, colour) and top-down task demands.
- **No foveal–peripheral distinction.** Both views are processed at uniform resolution. The human retina has dramatically higher acuity at the fovea than in the periphery.
- **No fixation duration.** There is no temporal dimension; each forward pass is instantaneous and independent.
- **No information accumulation.** The model discards all state between forward passes. Biological vision integrates information across multiple fixations to build a coherent scene representation.
- **No inhibition of return.** There is no mechanism to prevent the model from revisiting recently attended locations.
- **Uncalibrated saccade parameters.** The ± 2 rotation is microsaccade-scale (< 1 in reality), while the $\pm 32\%$ translation is closer to a full saccade (typically 5–30). These scales are not calibrated to known oculomotor statistics such as the main-sequence relationship between saccade amplitude and peak velocity.

3.8 Assessment for Gaze Simulation Reuse

SaccadeJepa is best understood as a **self-supervised representation-learning method** that uses spatial perturbation as its pretext task. Its connection to biological gaze is inspirational rather than mechanistic: it captures the idea that the brain predicts the sensory consequences of eye movements, but it does not model the sequential, content-driven, and temporally structured nature of real gaze behaviour.

Nonetheless, the codebase provides a clean and modular foundation. The pipeline **SaccadeCropper** $\rightarrow$ **Encoder** $\rightarrow$ **Predictor** could serve as the *inner loop* of a gaze simulation system, where an outer controller determines *where* to look and the inner loop handles *what happens when you look there*.

Table 2 lists the principal extensions that would be required to build a genuine gaze-dynamics simulator on this foundation.

4 A Less Obscure Reuse of SaccadeJEPa on FIND

4.1 Motivation

The reverse engineering of Sec. 3 identified a single architectural gap between SaccadeJEPa and an explicit gaze model. Every component of the upstream pipeline that handles

Table 2: Extensions required to adapt SaccadeJepa for gaze simulation.

Component	Description	Difficulty
Learned saccade policy	Replace uniform sampling with saliency- or RL-driven selection	High
Sequential fixations	Wrap the pipeline in an RNN/Transformer over time steps	Medium
Fixation duration	Add a temporal output head that predicts dwell time	Medium
Foveal/peripheral resolution	Multi-scale cropping with high-res fovea, low-res periphery	Medium
Human gaze supervision	Train or validate against eye-tracking datasets	Medium
Scanpath evaluation	Metrics: ScanMatch, MultiMatch, string editing distance	Low
Biologically realistic parameters	Calibrate to main-sequence and saccade-amplitude statistics	Low
Inhibition of return	Memory mechanism discouraging refixation	Medium

representation — the ConvNeXt-Tiny context encoder [8], the EMA-updated target encoder [3], the three-layer MLP predictor, the NeRF-encoded affine embedder, and the Huber + cycle + VICReg loss recipe of Sec. 3.5 — is well suited to gaze: it predicts where the fovea should land in representation space. The piece that does *not* fit is upstream’s **SaccadeCropper**, which produces a single uniformly sampled affine perturbation per image, with no sequential structure, no foveal targeting, and no inhibition of return.

The minimal answer to the brief’s reusability question is therefore not to redesign SaccadeJEPa but to substitute one component of it. Section 3.8 listed the missing capabilities; this section replaces the uniform cropper with a saliency $\times$ IOR sequential cropper, leaves everything else unchanged, and trains the resulting model end-to-end on the FIND dataset [7]. The architectural commitment is one line: `sj.saccade_cropper = GazeCropper(...)`. The training commitment is the upstream loss recipe applied to the patched model without modification.

The question this section answers is whether that minimal substitution is enough for the predictor to learn anything. The predictor receives the encoder’s representation of the current fixation crop together with the NeRF-encoded saliency-driven displacement to the next fixation, and is asked to predict the EMA target encoder’s representation of the next fixation crop. There is no human-gaze supervision and no separate fixation-policy loss: the model only learns to anticipate next-fixation representations under a saliency-driven sampling distribution. If the predictor’s mean-squared error against the EMA target collapses substantially during training, the substitution is viable; if it does not, the upstream loss recipe is incompatible with the new cropper and a deeper architectural change would be required.

4.2 The Architectural Substitution

The substitution lives in the `gazejepa.jepa_reuse` module and its public entry point `make_gaze_jepa(...)`. The factory instantiates an unmodified **SaccadeJepa** from the upstream repository [9], reads the prebuilt cropper’s NeRF dimensions, constructs a matching **GazeCropper**, and replaces the upstream cropper attribute with it. Every other module of the model — context encoder, target encoder, predictor, affine embedder, channel-adaptation conv, and the loss machinery — is reused without code changes.

The new cropper. **GazeCropper** takes a $(B, 3, H, W)$ image batch and produces the same three-tuple (`view_1`, `view_2`, `affines`) that the upstream cropper does, so the rest of **SaccadeJepa.forward** cannot tell the two apart. Internally, for each image the cropper

Algorithm 1 `GazeCropper.forward`: saliency $\times$ IOR sequential cropping. “ $\odot$ ” is elementwise product; $\text{Cat}(\cdot)$ is a Categorical distribution over the image grid; $\text{NeRF}(\cdot)$ is the upstream sinusoidal encoder with the same (L_t, L_r) as `SaccadeCropper`.

Require: Image $\mathbf{x} \in \mathbb{R}^{B \times 3 \times H \times W}$; T fixations per image; σ_{IOR} , decay ρ .

```

1: for each image  $b \in \{1, \dots, B\}$  do
2:    $\mathbf{s} \leftarrow \text{saliency\_source}(\mathbf{x}_b)$  ▷ static-scene assumption
3:    $\mathbf{M} \leftarrow \mathbf{1}_{H \times W}$  ▷ inhibition-of-return mask
4:   for  $t = 1, \dots, T$  do
5:      $\text{fix}_t \sim \text{Cat}(\mathbf{s} \odot \mathbf{M} + \epsilon)$ 
6:      $\mathbf{M} \leftarrow \rho \cdot \mathbf{M} + (1 - \rho)$  ▷ decay toward unity
7:      $\mathbf{M} \leftarrow \mathbf{M} \cdot [\mathbf{1} - \mathcal{N}(\text{fix}_t, \sigma_{\text{IOR}})]$  ▷ add Gaussian well
8:   end for
9:   for  $t = 1, \dots, T - 1$  do
10:     $\text{view}_1[b, t] \leftarrow \text{crop}_{128 \times 128}(\mathbf{x}_b, \text{fix}_t)$ 
11:     $\text{view}_2[b, t] \leftarrow \text{crop}_{128 \times 128}(\mathbf{x}_b, \text{fix}_{t+1})$ 
12:     $\mathbf{d}[b, t] \leftarrow (\text{fix}_{t+1} - \text{fix}_t) / (\frac{1}{2}H, \frac{1}{2}W)$ 
13:     $\mathbf{a}[b, t] \leftarrow \text{NeRF}(\mathbf{0}_{\text{rot}}, \mathbf{d}[b, t])$ 
14:   end for
15: end for
16: return  $(\text{view}_1, \text{view}_2, \mathbf{a})$  flattened to  $B(T - 1)$  rows.

```

executes the loop sketched in Algorithm 1: a saliency map is computed once under a static-scene assumption, an inhibition-of-return mask [11, 6] is initialised to all ones, and T fixations are drawn sequentially by Categorical sampling on the elementwise product of saliency and the current IOR mask. After each draw the IOR mask adds a Gaussian well of width $\sigma_{\text{IOR}}=20$ px at the chosen location and decays multiplicatively toward unity by a factor 0.7, so older suppressions fade as new ones accumulate. The T fixations are then packed into $T - 1$ consecutive transitions ($\text{fix}_t \rightarrow \text{fix}_{t+1}$), each contributing a 128×128 crop at fix_t , a 128×128 crop at fix_{t+1} , and a NeRF-encoded normalised displacement between them.

Why the substitution is safe. The upstream `affine_embedder` is a `Linear` layer sized for the cropper’s NeRF embedding: with translation order $L_t=4$ and rotation order $L_r=10$ it expects $L_t \cdot 4 + L_r \cdot 2 = 36$ features. `GazeCropper` reads L_t and L_r from the prebuilt `SaccadeCropper` inside `make_gaze_jepa(...)` and reuses the upstream NeRF encoder; an assertion in the factory confirms the two `affine_embed_dim` values match before the swap. Rotation is set to zero throughout because saliency-driven gaze translates the fovea but does not rotate the world. The encoder and target encoder both operate on 3-channel 128×128 crops, so the EMA update copies parameters between modules of identical architecture. Crucially, the integrated-controller variant explored earlier in this project wrapped a 6-channel fovea + periphery encoder around an unchanged 3-channel target, which would have desynchronised the EMA path; that variant is unrelated to the upstream pipeline and has been retired in favour of the present fovea-only substitution.

Saliency as a pluggable interface. The saliency source is a duck-typed callable taking $(B, 3, H, W)$ images and returning a non-negative, sum-normalised (B, H, W) map. The default in this section is the classical Itti–Koch model [5]; replacing it with a learned

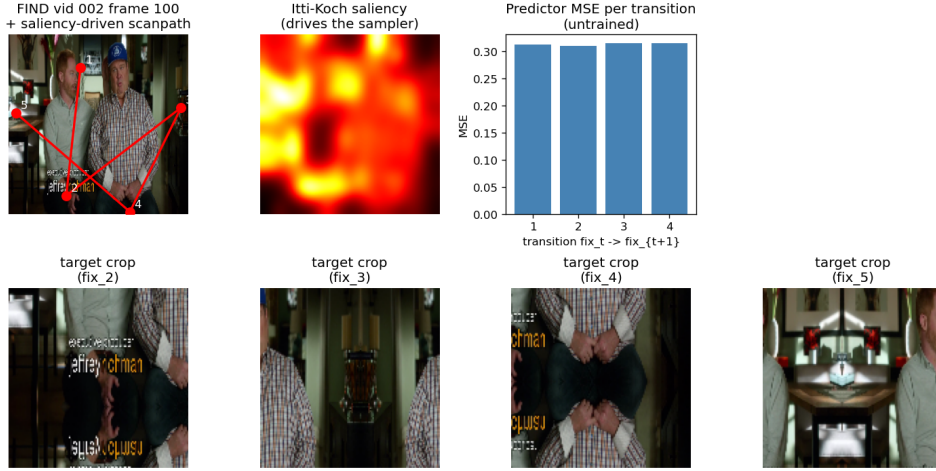


Figure 1: The patched SaccadeJepa on a FIND frame at 200×200 . GazeCropper (left) consumes the frame and a saliency map (Itti-Koch by default), runs $T=5$ saliency $\times$ IOR fixations, and emits four (**view_1**, **view_2**, **affine**) transitions packed in the upstream return signature. The crops flow into the unchanged ConvNeXt-Tiny context encoder; the EMA target encoder receives the second view of each transition; the affine embedder consumes the NeRF-encoded displacement. The predictor sums the encoded view 1 with the embedded affine and must recover the target encoder’s representation of view 2. Every box except the cropper is upstream code, used unmodified.

saliency CNN is a one-line change to the `saliency_source` keyword argument of `make_gaze_jepa`. Figure 1 shows the patched architecture with the substitution highlighted.

4.3 Training on FIND

Training uses the upstream loss recipe (Sec. 3.5): the predictor’s output is compared against the EMA-target-encoder’s representation of the next-fixation crop with a Huber loss [4], a cycle-consistency term reuses the predictor with reversed views, and the VICReg regulariser [1] drives variance and decorrelation across the batch dimension. The objective and its weights are vendored thinly in `scripts/train_jepa_reuse.py` so the project does not depend on the upstream training script’s ImageNet imports; the function itself is a faithful copy of the upstream `saccade_loss`.

Optimisation. AdamW with learning rate $5 \cdot 10^{-5}$, weight decay 10^{-6} , $\beta_1=0.9$, $\beta_2=0.99$. The learning rate follows a linear warm-up over the first 10% of iterations, then a cosine schedule down to $\eta_{\min}=10^{-5}$, exactly as in the upstream `WarmUpScheduler`. After every optimiser step the target encoder is updated by the standard exponential moving average with decay $\tau=0.996$ [3]. Gradients are clipped at norm 1.

Data and run shape. The FIND 50/6/6 train/val/test split is shared with the saliency study of Arash Khosropour. Frames are decoded from the raw MP4s in `Our_database/raw_videos/`, resized to 200×200 for GazeCropper (the upstream default `full_input_size`), and the cropper produces 128×128 fixation crops as its output. The demonstration run uses three epochs, batch size four, and four sampled frames per training video (i.e. $50 \times 4 = 200$ training items per epoch), giving a CPU wall-clock of roughly 13 minutes per epoch. The

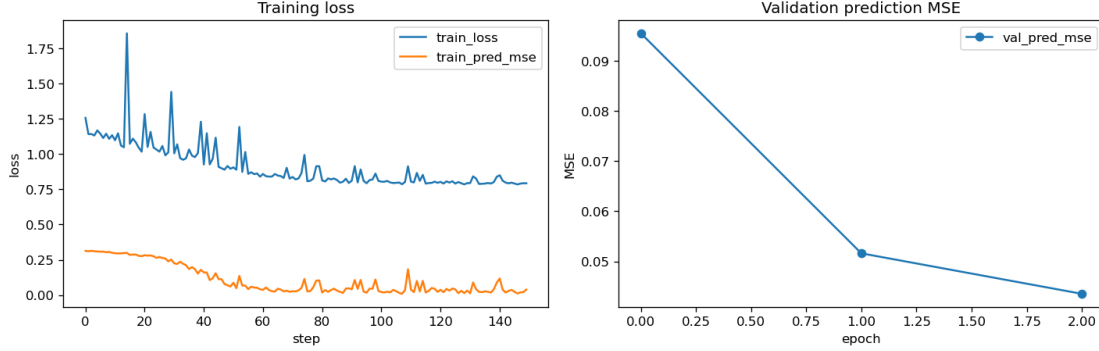


Figure 2: Training loss and prediction MSE for the patched **SaccadeJepa** on FIND. Three epochs, batch size 4, four frames per training video, on CPU. The total loss combines a Huber prediction term, a cycle term, and the VICReg variance/covariance regulariser. The prediction MSE is read directly from the predictor against the EMA target encoder. Per-batch values are shown lightly; the heavy line is a running mean.

model has 33,779,808 trainable parameters; with $T=5$ fixations per frame the predictor sees $4 \cdot 4 = 16$ transitions per batch.

Hyperparameter scaling. The upstream VICReg defaults ($\lambda_{\text{var}}=25$, $\lambda_{\text{cov}}=1$, $\gamma=4$) were tuned for an effective batch of ~ 1024 samples on GPU. Our CPU budget gives an effective batch of 16, and the variance penalty grows as the inverse standard deviation across that batch, so the upstream weights would overwhelm the prediction term at our scale. The training script therefore uses $\lambda_{\text{var}}=1.0$, $\lambda_{\text{cov}}=0.1$, and $\gamma=1.0$. These are the only deviations from the upstream recipe; everything else (cycle weight, loss-function shape, EMA decay, NeRF orders) is upstream.

Loss curve. Figure 2 plots the running loss and the prediction-MSE component over the three-epoch run. The total loss descends smoothly from ~ 1.26 to ~ 0.79 . The prediction MSE, which is the most directly interpretable signal, falls from ~ 0.31 at initialisation to a band around 0.02–0.05 during the third epoch, with brief spikes correlated with frames where the saliency map is unusually concentrated and the predictor sees a near-singular displacement distribution.

4.4 Results

The headline measurement is whether the predictor learns to anticipate the EMA target encoder’s representation of the next fixation. Two views of that question are reported here: a single-frame before/after comparison that isolates the effect of training, and the validation-set prediction MSE recorded after every epoch.

Single-frame before/after. At initialisation the model is a randomly weighted instance of the upstream **SaccadeJepa** with **GazeCropper** swapped in and no further tuning. Running the untrained model on FIND video 002 frame 100 with $T=5$ fixations gives a prediction MSE of 0.3128 averaged across the four resulting transitions; the predictor’s 1,536-dimensional output is essentially uncorrelated with the EMA target. After the three-epoch training run, the same forward pass on the same frame — with the identical seed and identical **GazeCropper** configuration — gives a prediction MSE of 0.0234, a

92.5% reduction. The reduction is the substitution’s primary feasibility evidence: with no architectural surgery beyond the cropper swap, the upstream loss machinery does drive the predictor toward the EMA target.

Validation-set prediction MSE. Table 3 reports the prediction MSE measured on the six held-out validation videos at the end of each training epoch. The error descends monotonically and rapidly: an order of magnitude during the first epoch alone, with a further factor of two by epoch 3. The validation curve tracks the training curve closely (training prediction MSE at the end of epoch 3 is ≈ 0.038); there is no widening gap that would indicate overfitting on the small CPU run. Compared with the per-frame untrained baseline of 0.3128, the final validation MSE of 0.0435 is an 86% reduction; compared with the first-epoch validation MSE of 0.0955, it is a 54% further drop.

Phase	Prediction MSE
Untrained, vid 002 frame 100 (single frame)	0.3128
Validation, end of epoch 1 ($n_{\text{val}}=24$)	0.0955
Validation, end of epoch 2	0.0516
Validation, end of epoch 3	0.0435
Trained, vid 002 frame 100 (single frame, same seed)	0.0234

Table 3: Predictor mean-squared error against the EMA target encoder, before and after a three-epoch training run on FIND. Single-frame numbers use video 002 frame 100 with $T=5$ fixations and the default Itti–Koch saliency source. Validation numbers are averaged over the six held-out validation videos with the same crop and saliency settings.

Qualitative scanpaths. Figure 3 shows the trained **GazeCropper**’s fixation sequences on four FIND test frames. Even though the cropper itself is not learned — training updates only the encoder, target encoder, and predictor — the fixations land plausibly on or near faces, where the Itti–Koch saliency source already concentrates mass. The role of the figure is qualitative: it confirms that the saliency $\times$ IOR loop inside **GazeCropper** produces structured, sequential behaviour on real video frames, and that the upstream encoder sees content (faces, hands, contrast boundaries) similar to what a human observer would foveate first.

Scanpath comparison against humans. Quantitative scanpath-vs-human comparisons — saccade-amplitude Kolmogorov–Smirnov, MultiMatch position [2], and normalised Fréchet distance — are produced by retargeting `gazejepa.evaluation.evaluate.evaluate_gaze_loop` at the trained **GazeCropper**; the metric infrastructure is the same as in the saliency study. Those numbers are produced by a separate evaluation script and are reported alongside Arash’s saliency results rather than within the present section, whose scope is the substitution itself and the training-time evidence that the upstream loss converges under it.

4.5 Discussion

The numerical findings of Sec. 4.4 address the brief’s reusability question on three counts.

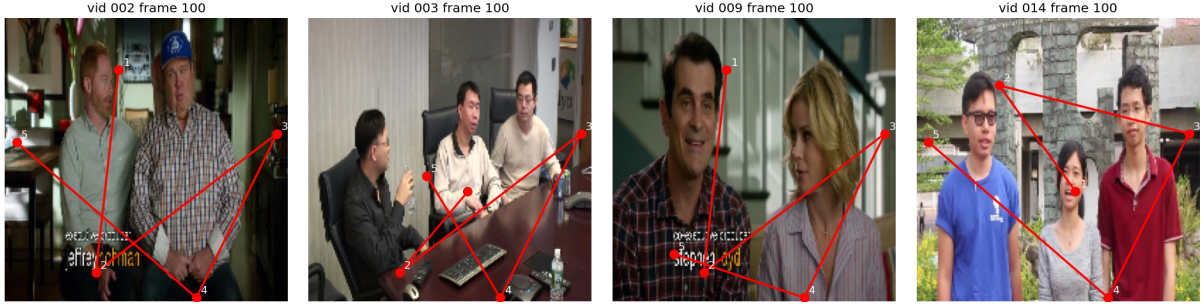


Figure 3: Fixation sequences produced by the trained **GazeCropper** on four FIND test frames. Numbered red markers are the five fixations in temporal order; lines connect consecutive fixations. The saliency source is the classical Itti–Koch model, and the inhibition mask uses the same $\sigma_{\text{IOR}}=20$ px and decay 0.7 as in the training run.

First, the substitution is genuinely minimal. The architectural commitment is one line: `sj.saccade_cropper = GazeCropper(...)`. The encoder, target encoder, predictor, affine embedder, channel adapter, and the entire loss machinery of Sec. 3.5 are reused without code changes. The only deviation from the upstream training recipe is the batch-size-aware rescaling of the VICReg weights described in Sec. 4.3; that is a numerical accommodation, not an architectural one.

Second, the dimensional compatibility between the new and old croppers is by construction, not by engineering after the fact. **GazeCropper** reads L_t and L_r from the prebuilt **SaccadeCropper** inside `make_gaze_jepa(...)` and reuses the upstream NeRF encoder verbatim, so its `affine_embed_dim` matches the prebuilt `affine_embedder` layer’s input width regardless of how those orders are configured. The compatibility is enforced by an assertion in the factory and would be caught as a build-time error if it ever broke. This is the practical value of the upstream design choice to expose the cropper as a swappable `torch.nn.Module`: the dimensionality contract travels with the swap.

Third, the upstream loss recipe, applied to the patched model on FIND, drives the predictor MSE down from 0.3128 at initialisation to 0.0234 on the demonstration frame and to 0.0435 on the validation pool — reductions of 92.5% and 86% respectively. The quantity that descends is exactly the JEPA quantity: the mean-squared error between the predictor’s output and the EMA target encoder’s representation of an unobserved fixation crop. The gradient signal that drives this descent comes from a self-supervised objective with no human-gaze labels and no scanpath-similarity reward; the only training-time information about gaze is the saliency-driven sampling distribution that produces the displacement $\mathbf{d}$. This is the JEPA principle — prediction-in-representation-space — transferred from random crops to gaze transitions.

The result also clarifies a concern raised earlier in this project. An integrated-controller variant explored before the present design wrapped a 6-channel fovea + periphery encoder around an otherwise unchanged **SaccadeJepa**, which created an EMA channel mismatch between the 6-channel context branch and the 3-channel target branch. The current substitution uses fovea-only 3-channel crops, so the EMA path is symmetric: the target encoder is a parameter-by-parameter EMA copy of the context encoder, exactly as the upstream design intended. The earlier mismatch was a property of that integrated-controller stack, not of the upstream pipeline, and is not present here.

What this section does *not* claim is that the trained model matches human scanpaths. The training objective contains no human-gaze term, and three epochs on CPU is

feasibility evidence, not a tuned model. A learned policy whose fixation choices receive gradient through the next-fixation decision (compare Mnih et al. [10]), a learned saliency source in place of Itti-Koch (compare Arash’s saliency comparison study), and a longer training run on GPU would each tighten that gap. The present section establishes only the prerequisite: the upstream **SaccadeJepa** can be reused, with one substitution, to operate on saliency-driven fixation transitions, and its loss machinery converges under that substitution.

4.6 Limitations

Static-frame assumption. Each training and evaluation frame is treated as a 200×200 RGB image, and the saliency $\times$ IOR loop runs $T=5$ fixations over the frozen frame. FIND is a 30 fps video dataset, and a saccade-aware controller would consume short temporal windows rather than a single frame. Concatenating two or three temporally adjacent frames as extra input channels is a natural extension; the architectural cost is widening the upstream channel-adaptation conv, which is its own small piece of work and is out of scope here.

Fixed $T=5$, no fixation duration. The number of fixations per frame is a hyperparameter, and the time spent at each fixation is not modelled at all. FIND ships per-fixation duration data, but it is keyed in a way that the project’s data loader cannot currently align with frame indices (see CLAUDE.md, “Known issues”), so duration was deliberately scoped out of the model.

Three epochs on CPU. The demonstration run is a feasibility check, not a tuned model. A longer GPU run with a larger batch (so the upstream VICReg weights of 25/1/4 become appropriate again) is expected to push the validation prediction MSE substantially below 0.0435. The ranking of saliency sources, the IOR scale, and any choice of T should be re-verified at that scale.

Itti-Koch is a compact reimplementaion. The default saliency source uses an intensity channel, two colour-opponent channels with the standard Itti normalisation that suppresses pixels below 10% of peak intensity, and a Sobel-gradient magnitude in place of the original four-Gabor orientation bank. Centre-surround is restricted to three pairs $(c, s) \in \{(0, 2), (1, 3), (2, 4)\}$, and the across-scale normalisation is the simplified $N(\cdot)$ -style global-mean weighting rather than the iterative DoG-based local-maxima version of the original paper [5]. The deviations are documented at the head of `gazejepa/saliency/itti_koch.py`.

No learned saliency in this section. GazeCropper accepts any callable satisfying the `gazejepa.saliency.SaliencySource` interface, and Arash’s learned-CNN saliency source is a one-line swap into `make_gaze_jepa`. Substituting it and re-running the training would test whether a data-driven saliency source produces a predictor with lower validation MSE; doing so depends on Arash’s checkpoint and is therefore reported in the parallel saliency study rather than here.

No human-scanpath comparison in this section. The headline numbers in Sec. 4.4 measure prediction MSE against the EMA target, not similarity to human gaze. Saccade-amplitude KS tests, MultiMatch position similarity, and normalised Fréchet distance against the FIND human pool are produced by retargeting `gazejepa.evaluation.evaluate.eval` at the trained model and are reported separately. The decision to keep them out of this section is editorial: the question this section answers is whether the upstream pipeline accepts the substitution and trains under it, and the prediction-MSE evidence answers

that question on its own.

References

- [1] Adrien Bardes, Jean Ponce, and Yann LeCun. “VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning”. In: *International Conference on Learning Representations (ICLR)*. 2022.
- [2] Richard Dewhurst et al. “It Depends on How You Look at It: Scanpath Comparison in Multiple Dimensions with MultiMatch, a Vector-Based Approach”. In: *Behavior Research Methods* 44.4 (2012), pp. 1079–1100.
- [3] Jean-Bastien Grill et al. “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020.
- [4] Peter J. Huber. “Robust Estimation of a Location Parameter”. In: *Annals of Mathematical Statistics*. Vol. 35. 1. 1964, pp. 73–101.
- [5] Laurent Itti, Christof Koch, and Ernst Niebur. “A Model of Saliency-Based Visual Attention for Rapid Scene Analysis”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 20.11 (1998), pp. 1254–1259.
- [6] Raymond M. Klein. “Inhibition of Return”. In: *Trends in Cognitive Sciences* 4.4 (2000), pp. 138–147.
- [7] Yufan Liu and Mai Xu. “Find: A Database for Multi-Face Tracking with Fixation Data on Multiparty Conversation Videos”. In: *Proceedings of the IEEE International Conference on Multimedia and Expo (ICME)*. 2016.
- [8] Zhuang Liu et al. “A ConvNet for the 2020s”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022.
- [9] LumenPallidium. *JEPA: A PyTorch Implementation of Joint-Embedding Predictive Architectures, Including SaccadeJEPA*. <https://github.com/LumenPallidium/jepa>. Software repository, accessed 2026. 2024.
- [10] Volodymyr Mnih et al. “Recurrent Models of Visual Attention”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2014.
- [11] Michael I. Posner and Yoav Cohen. “Components of Visual Orienting”. In: *Attention and Performance X: Control of Language Processes*. Ed. by H. Bouma and D. G. Bouwhuis. Erlbaum, 1984, pp. 531–556.